



About Jamboree Education:

Jamboree has helped thousands of students like you make it to top colleges abroad. Be it GMAT, GRE or SAT, their unique problem-solving methods ensure maximum scores with minimum effort. They recently launched a feature where students/learners can come to their website and check their probability of getting into the IVY league college. This feature estimates the chances of graduate admission from an Indian perspective.

Goals / Objectives:

1. Our analysis will help Jamboree in understanding what factors are important in graduate admissions.
2. We need to find how these factors are interrelated among themselves.
3. We need to create a model which can help in prediction of one's chances of admission given the rest of the variables.

```
In [41]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, mean
from sklearn.linear_model import Lasso, Ridge, LinearRegression
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
import statsmodels.stats.api as sms
from statsmodels.compat import lzip

import seaborn as sns
import warnings

with warnings.catch_warnings():
    warnings.simplefilter("ignore")
```

```
In [42]: df = pd.read_csv("/content/Jamboree_Admission.csv")
df.head(5)
```

```
Out[42]:
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	1	337	118	4	4.5	4.5	9.65	1	0.92
1	2	324	107	4	4.0	4.5	8.87	1	0.76
2	3	316	104	3	3.0	3.5	8.00	1	0.72
3	4	322	110	3	3.5	2.5	8.67	1	0.80
4	5	314	103	2	2.0	3.0	8.21	0	0.65

```
In [43]: df.shape
```

```
Out[43]: (500, 9)
```

```
In [44]: df.dtypes
```

```
Out[44]:
```

	0
Serial No.	int64
GRE Score	int64
TOEFL Score	int64
University Rating	int64
SOP	float64
LOR	float64
CGPA	float64
Research	int64
Chance of Admit	float64

dtype: object

```
In [45]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Serial No.            500 non-null   int64
1   GRE Score              500 non-null   int64
2   TOEFL Score            500 non-null   int64
3   University Rating      500 non-null   int64
4   SOP                    500 non-null   float64
5   LOR                    500 non-null   float64
6   CGPA                   500 non-null   float64
7   Research               500 non-null   int64
8   Chance of Admit        500 non-null   float64
dtypes: float64(4), int64(5)
memory usage: 35.3 KB
```

```
In [46]: df.isna().sum()
```

```
Out[46]:
```

Serial No.	0
GRE Score	0
TOEFL Score	0
University Rating	0
SOP	0
LOR	0
CGPA	0
Research	0
Chance of Admit	0

dtype: int64

- From above analysis we can see that there are no null values in any of the columns in our dataset

Checking for duplicates

```
In [47]: df.duplicated().sum()
```

```
Out[47]: np.int64(0)
```

Quick summary statistics for numerical columns

```
In [48]: df.describe()
```

```
Out[48]:
```

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500
mean	250.500000	316.472000	107.192000	3.114000	3.374000	3.484000	
std	144.481833	11.295148	6.081868	1.143512	0.991004	0.925450	
min	1.000000	290.000000	92.000000	1.000000	1.000000	1.000000	
25%	125.750000	308.000000	103.000000	2.000000	2.500000	3.000000	
50%	250.500000	317.000000	107.000000	3.000000	3.500000	3.500000	
75%	375.250000	325.000000	112.000000	4.000000	4.000000	4.000000	
max	500.000000	340.000000	120.000000	5.000000	5.000000	5.000000	

Defining the Numerical and Categorical column sets

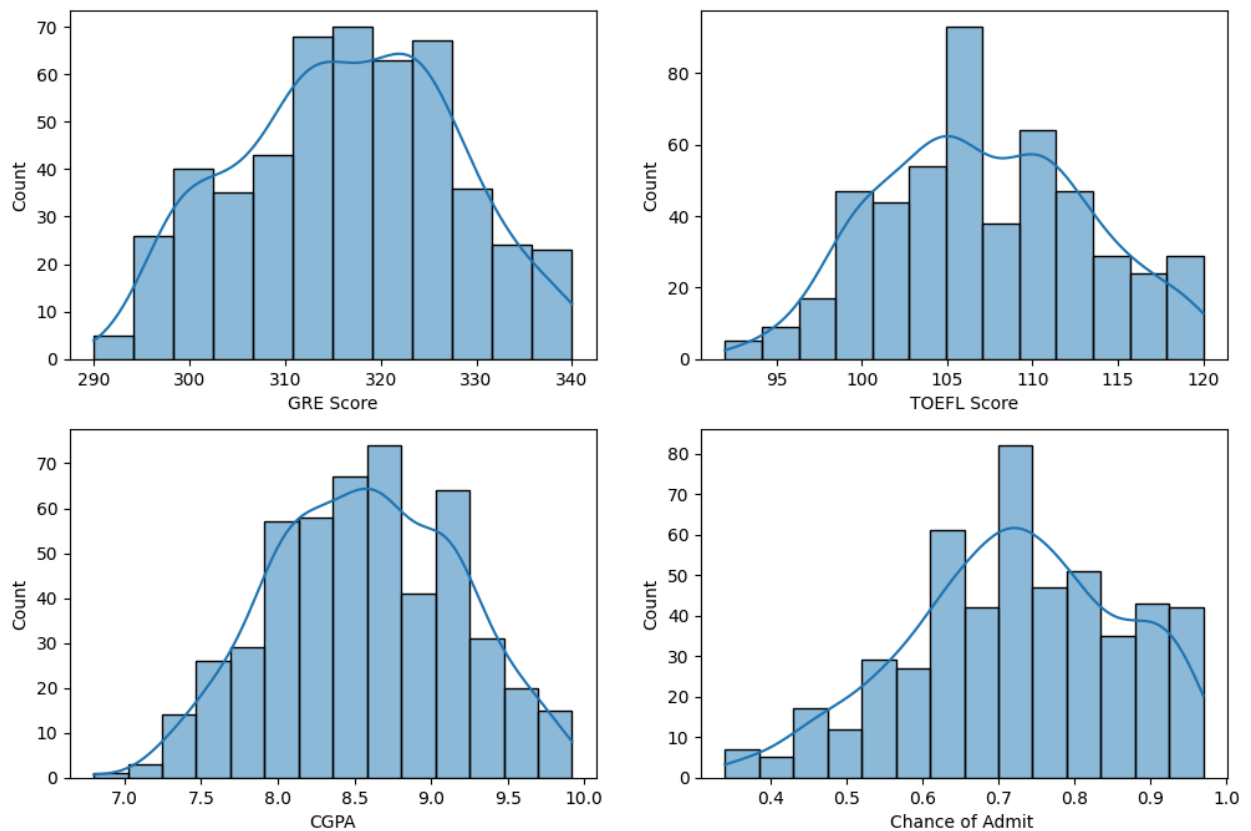
```
In [49]: num_col = ['GRE Score', 'TOEFL Score', 'CGPA']  
        categ_col = ['University Rating', 'SOP', 'LOR ', 'Research']  
        target_col = 'Chance of Admit '
```

Univariate Analysis of numerical columns:

```
In [50]: df.columns
```

```
Out[50]: Index(['Serial No.', 'GRE Score', 'TOEFL Score', 'University Rating', 'SOP',  
               'LOR ', 'CGPA', 'Research', 'Chance of Admit '],  
              dtype='object')
```

```
In [51]: all_num_col = ['GRE Score', 'TOEFL Score', 'CGPA', 'Chance of Admit '] # Added  
        rows, cols = 2, 2  
        index = 0  
  
        fig, axs = plt.subplots(rows, cols, figsize=(12,8))  
  
        for row in range(rows):  
            for col in range(cols):  
                if index < len(all_num_col): # Add this condition to check if index is within  
                    sns.histplot(df[all_num_col[index]], kde=True, ax=axs[row, col])  
                    index += 1  
                else:  
                    axs[row, col].axis('off') # Turn off the extra subplot if no more columns  
  
        plt.show()
```



Analysis:

- We can see almost all numerical columns are roughly (with some minor ups and downs) following a normal distribution like curve centered around the mean.
- The left-skew for target column (Chance of Admit) indicates that relatively fewer candidates have a low chance of admission, while the majority have moderate to high chances.
- The majority of applicants in the dataset have a greater than 50% chance of admission, according to the distribution shown

BOX

```
In [52]: all_num_col = ['GRE Score', 'TOEFL Score', 'CGPA', 'Chance of Admit '] # Added
rows, cols = 2, 2
index = 0

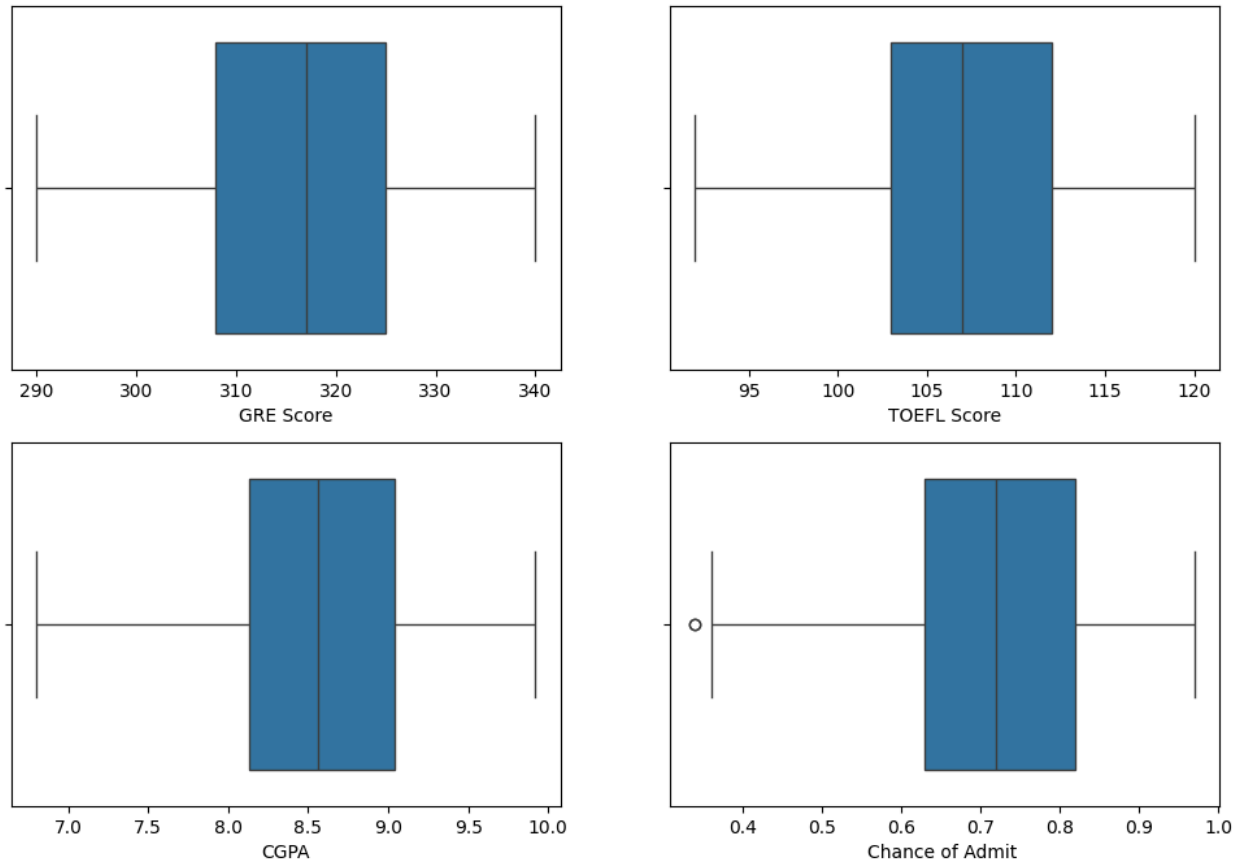
fig, axs = plt.subplots(rows, cols, figsize=(12,8))

for row in range(rows):
    for col in range(cols):
        if index < len(all_num_col): # Add this condition to check if index is wit
```

```

sns.boxplot(df[all_num_col[index]], ax=axis[row, col], orient='h')
index += 1
else:
    axis[row, col].axis('off') # Turn off the extra subplot if no more columns
plt.show()

```



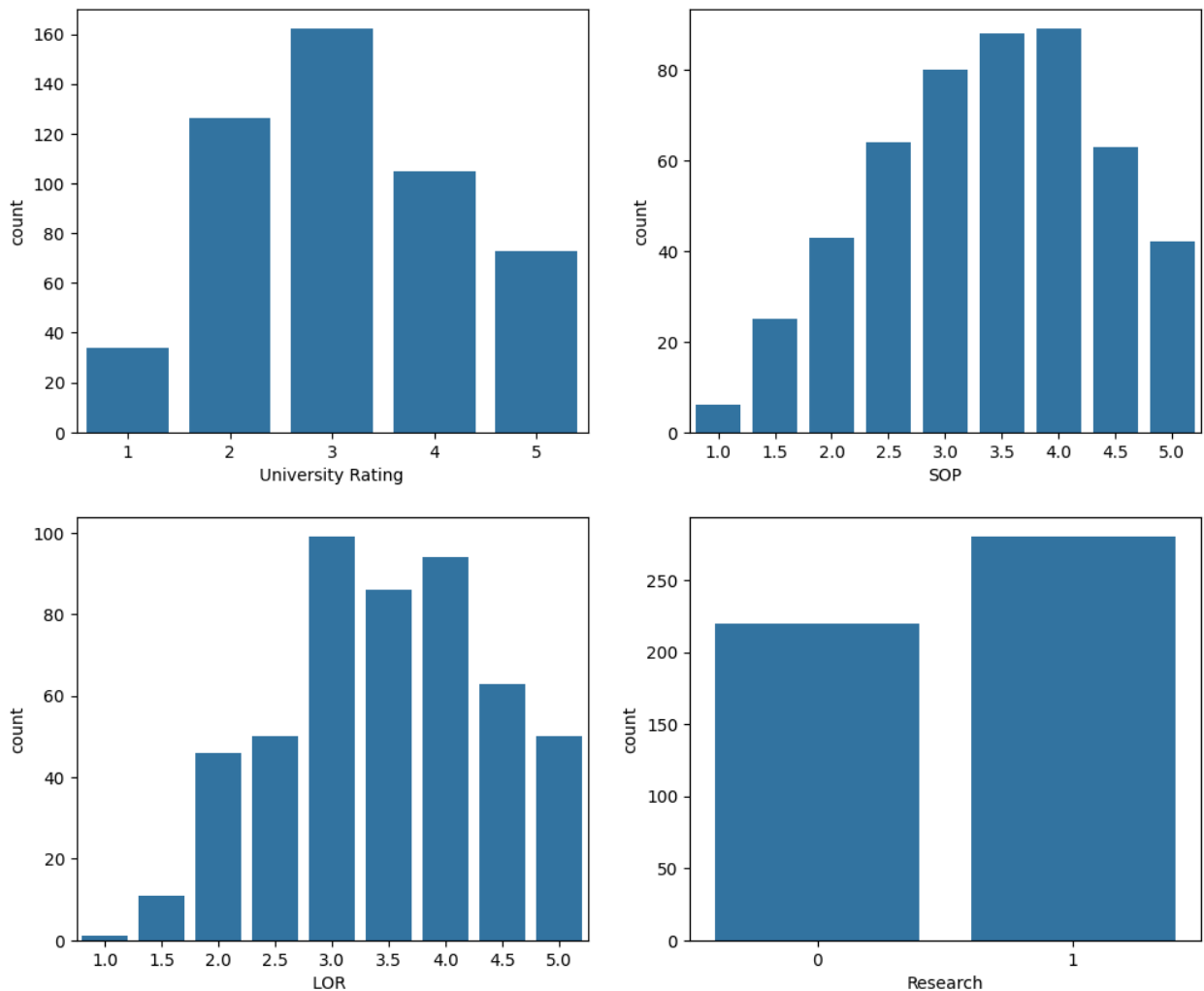
Analysis:

- Features like GRE, TOEFL and CGPA are roughly normally distributed without any skewness (As seen in KDE analysis) that's the reason we can't see any outliers for these columns.
- Chance of Admit column has a left skewed distribution (As seen in KDE analysis) which is further getting validated by presence of outliers in its data, because presence of outliers pull the mean towards itself which results in skewness.
- Almost all the columns have their median mostly around mean, because of no or very less presence of outliers.

Univariate Analysis for categorical columns:

```
In [53]: rows=2
cols=2
index=0
fig, axes = plt.subplots(rows, cols, figsize=(12,10))

for row in range(rows):
    for col in range(cols):
        if index < len(categ_col):
            sns.countplot(x=categ_col[index], data=df, ax=axes[row, col])
            index += 1
        else:
            axes[row, col].axis("off")
plt.show()
```



Analysis:

- **Most Common University Rating:** The top-left bar plot shows the

distribution of university ratings. The tallest bar indicates which rating is most frequent in the dataset. In the count plot, "3" is the most common university rating.

- **Most Common Statement of Purpose (SOP) and Letter of Recommendation (LOR) Strength:**

1. The top-right plot shows SOP ratings. The highest bars at "3.5" and "4.0" suggest these are the most common SOP strengths.
2. The bottom-left plot shows LOR strengths. Here, ratings of "3.0" and "4.0" appear most frequently, so these are the most common LOR strengths.

- **Whether Most People Have Done Research:** The bottom-right plot represents the "Research" column (0 = No, 1 = Yes). The bar for "1" (Research done) is higher, indicating that majority people in the dataset have done research.

Bivariate Analysis

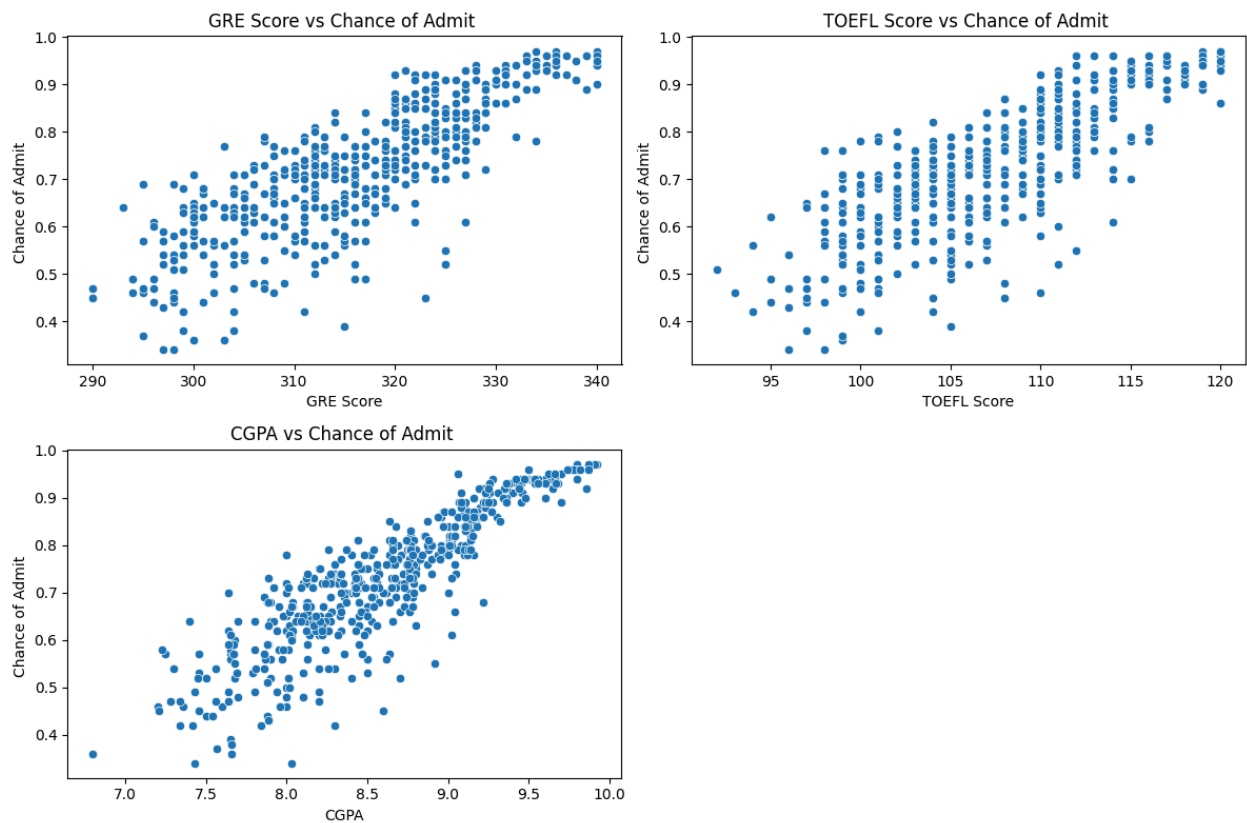
Let's explore the relationships between different features and the target variable ('Chance of Admit').

```
In [54]: rows, cols = 2, 2
         index = 0

         fig, axs = plt.subplots(rows, cols, figsize=(12,8))

         for row in range(rows):
             for col in range(cols):
                 if index < len(num_col):
                     sns.scatterplot(x=df[num_col[index]], y=df[target_col], ax=axs[row, col])
                     axs[row, col].set_title(f'{num_col[index]} vs {target_col}')
                     index += 1
                 else:
                     axs[row, col].axis('off')

         plt.tight_layout()
         plt.show()
```

Analysis:

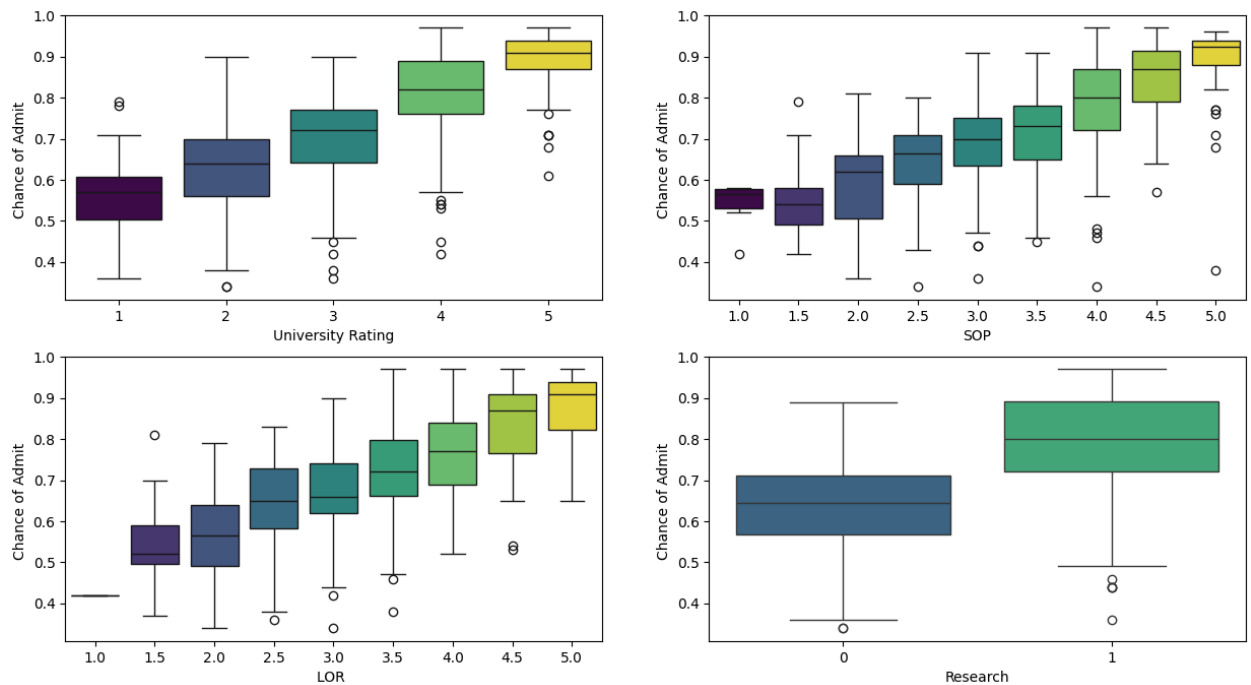
- Above Scatter plot analysis gives us an understanding of Linear relationship between predictor and target variable
- In all the plots as the GRE/TOEFL/CGPA score increases, the chance of admit also generally increases.

Bivariate Analysis between categorical columns and target column

```
In [55]: rows, cols = 2, 2
         index = 0

         fig, axs = plt.subplots(rows, cols, figsize=(15,8))

         for row in range(rows):
             for col in range(cols):
                 if index < len(categ_col): # Add this condition to check if index is within
                     sns.boxplot(x = df[categ_col[index]], y = df[target_col], hue=df[categ_col[index]],
                                   index += 1
                     else:
                         axs[row, col].axis('off') # Turn off the extra subplot if no more columns
         plt.show()
```



Analysis:

- **Outliers:** The circles beyond the "whiskers" of each box plot in your image represent outlier values for Chance of Admit within each category bin. These are data points that are unusually high or low compared to the rest in that group.
- **Research Experience → Higher Mean Probability:** The box for candidates with research (Research=1) is clearly shifted up—their median and range of probabilities are higher than those without research (Research=0). This means people with research experience on average have a higher predicted probability of admission.
- **Higher LOR/SOP → Higher Mean Probability:** The plots show that students with stronger letters of recommendation (LOR), and stronger statements of purpose (SOP) consistently have higher median (and typically mean) values for Chance of Admit—meaning these factors are positively associated with a better chance of admission in your data.
- **University Rating vs. Chance of Admit:** Students applying to higher-rated universities tend to have a higher chance of admission, Can We Say "People Applying to Higher University Ratings Have a Higher Chance of Admission"?

--> Yes, with caveats: The plot clearly shows a positive correlation—applicants to higher-rated universities in your data have

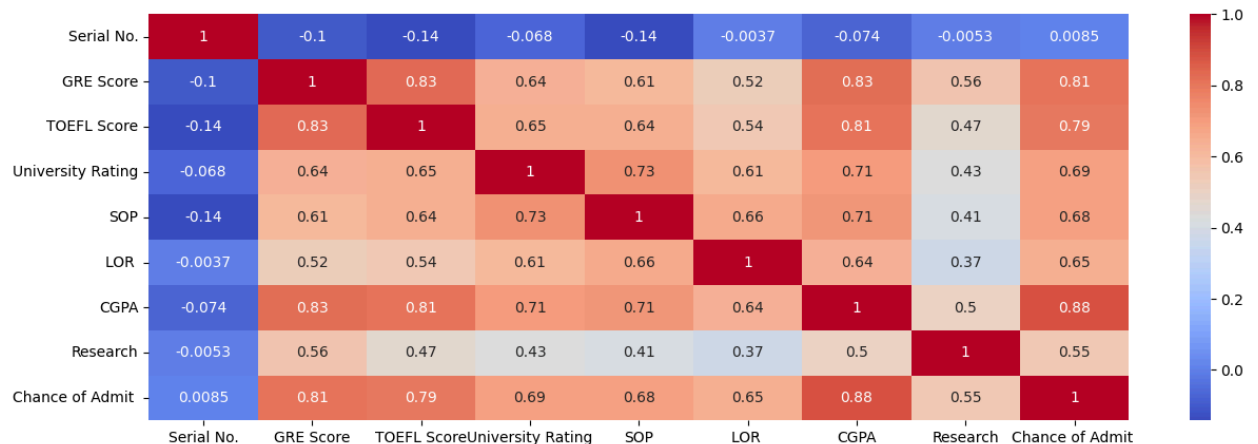
systematically higher predicted admission chances. This aligns with real-world intuition: Prestigious universities might attract stronger applicants (with better GRE/TOEFL/CGPA), leading to higher modeled probabilities (This is an association, not causation. The higher chances could be driven by other factors (e.g., self-selection of top students applying to top schools, or correlations with features like research experience)).

Multivariate Analysis

Correlation Matrix (Heat Map)

```
In [56]: fig = plt.figure(figsize=(15,5))
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")
```

Out[56]: <Axes: >



Analysis:

- Above plot represent the Pearson correlation coefficient between each pair of variables.
- **Interpretation:** The correlation coefficient measures how strongly two variables are related to each other—in terms of a linear relationship.
- A value of 0.88 means there is a strong positive linear relationship between CGPA and Chance of Admit (Target variable). As CGPA increases, the Chance of Admit tends to increase strongly as well, However, it does not mean "88% of the change in Chance of Admit is due to CGPA," nor does it mean they are "88% causally related"—it only

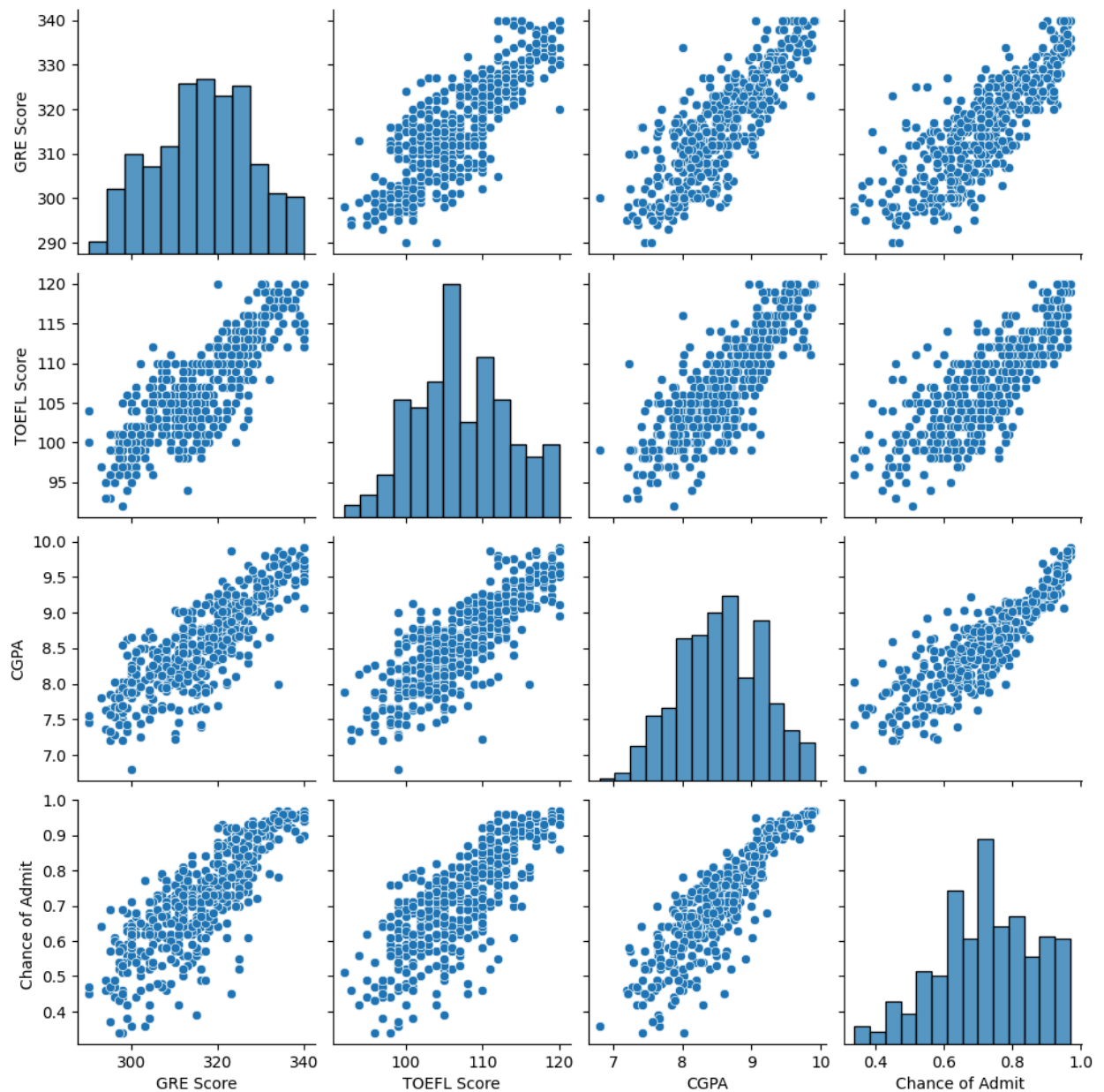
quantifies how closely their values move together linearly.

- Higher values (close to 1) between pairs mean those columns are strongly correlated. Example: GRE Score and TOEFL Score have a correlation of 0.83—meaning students with a higher GRE also tend to have a higher TOEFL score.
- **The correlation values on the heatmap help you:**
 1. **Detect multicollinearity:** Highly correlated columns might cause issues in linear regression (especially if >0.8).
 2. **Select important predictors:** Columns with higher correlation to the target (Chance of Admit) are good predictors.

Pairplot between numerical columns and target column to get a broad visual overview

```
In [57]: fig = plt.figure(figsize=(15, 5))  
sns.pairplot(df[num_col + [target_col]])
```

```
Out[57]: <seaborn.axisgrid.PairGrid at 0x7abfa9c03690>  
<Figure size 1500x500 with 0 Axes>
```



Data Preprocessing

Removing unnecessary columns

```
In [58]: df = df.drop(columns=['Serial No.'], axis=1)
df.columns
```

```
Out[58]: Index(['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR ', 'CGP
A',
               'Research', 'Chance of Admit '],
              dtype='object')
```

Checking duplicates

```
In [59]: df.duplicated().sum()
```

```
Out[59]: np.int64(0)
```

Defining predictor and target columns

```
In [60]: X = df.drop(columns=['Chance of Admit '])  
Y = df[target_col]
```

Scaling the data

```
In [61]: scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Dividing train and test data

```
In [62]: X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, Y, test_size=0.3  
print(X_train.shape, Y_train.shape)  
print(X_test.shape, Y_test.shape)  
  
(350, 7) (350,)  
(150, 7) (150,)
```

Important functions

```
In [63]: def adjusted_r2(r2, d, m):  
    adjr2 = 1 - ((1-r2)*(m-1) / (m-d-1))  
    return adjr2  
  
def getting_metrics(y_actual, y_pred, d):  
    mae = mean_absolute_error(y_true=y_actual, y_pred=y_pred)  
    mse = mean_squared_error(y_true = y_actual, y_pred = y_pred)  
    r2 = r2_score(y_true=y_actual, y_pred=y_pred)  
    adjr2 = adjusted_r2(r2, d, y_actual.shape[0])  
    rmse = root_mean_squared_error(y_true = y_actual, y_pred = y_pred)  
  
    response = {  
        "mae" : round(mae, 2),  
        "mse" : round(mse, 2),  
        "r2" : round(r2, 2),  
        "adjr2" : round(adjr2, 2),  
        "rmse": round(rmse, 2)  
    }  
  
    return response
```

Part 1: Model training (Model performance evaluation, without testing the assumptions of the linear regression model)

```
In [64]: def model_training(x_train, y_train, x_test, y_test, cols, use_model, alpha=1.  
        model = None  
        if use_model == "lasso":  
            model = Lasso(alpha=alpha)  
        elif use_model == "ridge":  
            model = Ridge(alpha=alpha)  
        else:  
            model = LinearRegression()  
  
        model.fit(x_train, y_train)  
        y_train_pred = model.predict(x_train)  
        y_test_pred = model.predict(x_test)  
  
        train_metrics = getting_metrics(y_train, y_train_pred, x_train.shape[1])  
        test_metrics = getting_metrics(y_test, y_test_pred, x_test.shape[1])  
  
        print(f"-----{use_model.title()} Regression Model-----")  
        print(f"Train MAE: {train_metrics['mae']}, Test MAE: {test_metrics['mae']}")  
        print(f"Train MSE: {train_metrics['mse']}, Test MSE: {test_metrics['mse']}")  
        print(f"Train R-Square: {train_metrics['r2']}, Test R-Square: {test_metrics['r2']}")  
        print(f"Train Adjusted R Square: {train_metrics['adjr2']}, Test Adjusted R Square: {test_metrics['adjr2']}")  
        print(f"Train RMSE: {train_metrics['rmse']}, Test RMSE: {test_metrics['rmse']}")  
        coef_df = pd.DataFrame({"Column": cols, "coef": model.coef_})  
        print(coef_df)  
        print(f"Intercept: {model.intercept_}")  
        print("=====")
```

```
In [65]: df.columns[:-1]
```

```
Out[65]: Index(['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR ', 'CGP  
A',  
              'Research'],  
              dtype='object')
```

```
In [66]: model_training(X_train, Y_train, X_test, Y_test, df.columns[:-1], "linear")  
model_training(X_train, Y_train, X_test, Y_test, df.columns[:-1], "lasso", alp  
model_training(X_train, Y_train, X_test, Y_test, df.columns[:-1], "ridge")
```

-----Linear Regression Model-----

Train MAE: 0.04, Test MAE: 0.04

Train MSE: 0.0, Test MSE: 0.0

Train R-Square: 0.82, Test R-Square: 0.82

Train Adjusted R Square: 0.82, Test Adjusted R Square: 0.81

Train RMSE: 0.06, Test RMSE: 0.06

	Column	coef
0	GRE Score	0.018657
1	TOEFL Score	0.023176
2	University Rating	0.011565
3	SOP	-0.000999
4	LOR	0.012497
5	CGPA	0.064671
6	Research	0.013968

Intercept: 0.724978121476996

=====

-----Lasso Regression Model-----

Train MAE: 0.04, Test MAE: 0.04

Train MSE: 0.0, Test MSE: 0.0

Train R-Square: 0.82, Test R-Square: 0.82

Train Adjusted R Square: 0.82, Test Adjusted R Square: 0.81

Train RMSE: 0.06, Test RMSE: 0.06

	Column	coef
0	GRE Score	0.018671
1	TOEFL Score	0.022770
2	University Rating	0.010909
3	SOP	0.000000
4	LOR	0.011752
5	CGPA	0.064483
6	Research	0.013401

Intercept: 0.7249659139557142

=====

-----Ridge Regression Model-----

Train MAE: 0.04, Test MAE: 0.04

Train MSE: 0.0, Test MSE: 0.0

Train R-Square: 0.82, Test R-Square: 0.82

Train Adjusted R Square: 0.82, Test Adjusted R Square: 0.81

Train RMSE: 0.06, Test RMSE: 0.06

	Column	coef
0	GRE Score	0.018902
1	TOEFL Score	0.023252
2	University Rating	0.011594
3	SOP	-0.000798
4	LOR	0.012539
5	CGPA	0.064004
6	Research	0.013990

Intercept: 0.7249823645841696

=====

Observation:

- The above results clearly states that there is no overfitting happening in the training of dataset because the adjusted R-square like

parameters are almost same for all Linear, Lasso and Ridge Regression

- The similarity between adjusted R-squared and R-squared suggests that the model does not contain redundant or non-informative predictors.

That means all the included variables are contributing meaningful information to the model's explanatory power.

Testing the assumptions of the linear regression model

Multicollinearity

```
In [67]: vif_threshold = 5
adj_r2_threshold = 0.79
updated_cols = df.columns[:-1]
features_removed = []
i = 0

while(True):
    primary_dataframe = pd.DataFrame(df.iloc[:, : -1], columns= df.columns[:-1])
    vif = pd.DataFrame()
    vif['features'] = updated_cols
    vif['vif_values'] = [variance_inflation_factor(primary_dataframe.values, i)
                        for i in updated_cols]
    vif['vif_values'] = round(vif['vif_values'], 2)

    vif = vif.sort_values(by="vif_values", ascending=False)
    print(vif)

    highest_vif_feature = vif.loc[0, "features"]
    highest_vif_value = vif.loc[0, "vif_values"]
    updated_cols = [i for i in updated_cols if i != highest_vif_feature]

    secondary_dataframe = pd.DataFrame(df.iloc[:, : -1], columns= df.columns[:-1])
    secondary_dataframe_sm = sm.add_constant(secondary_dataframe)
    OLS_model = sm.OLS(df.iloc[:, -1:], secondary_dataframe_sm.values).fit()
    print(f"Coloumn about to get removed {highest_vif_feature}")
    print(f"Updated Adjusted R Square will be {OLS_model.rsquared_adj}")

    if OLS_model.rsquared_adj < adj_r2_threshold or highest_vif_value > vif_threshold:
        print(f"threshold reached...")
        print(f"We can't remove {highest_vif_feature} and further columns")
        print(f"Following features were removed: {features_removed}")
        print(f"We removed {i} features")
        break

    features_removed.append(highest_vif_feature)
    i += 1
```

	features	vif_values
0	GRE Score	1308.06
1	TOEFL Score	1215.95
5	CGPA	950.82
3	SOP	35.27
4	LOR	30.91
2	University Rating	20.93
6	Research	2.87

Coloumn about to get removed GRE Score
Updated Adjusted R Square will be 0.8147176994385308

	features	vif_values
4	CGPA	728.78
0	TOEFL Score	639.74
2	SOP	33.73
3	LOR	30.63
1	University Rating	19.88
5	Research	2.86

Coloumn about to get removed TOEFL Score
Updated Adjusted R Square will be 0.8040281323797382

	features	vif_values
1	SOP	33.63
2	LOR	30.36
3	CGPA	25.10
0	University Rating	19.78
4	Research	2.84

Coloumn about to get removed University Rating
Updated Adjusted R Square will be 0.8022363642496624

	features	vif_values
1	LOR	29.36
0	SOP	25.74
2	CGPA	25.01
3	Research	2.74

Coloumn about to get removed SOP
Updated Adjusted R Square will be 0.8014400299294139

	features	vif_values
0	LOR	22.22
1	CGPA	20.79
2	Research	2.62

Coloumn about to get removed LOR
Updated Adjusted R Square will be 0.7921290293615955

	features	vif_values
0	CGPA	2.46
1	Research	2.46

Coloumn about to get removed CGPA
Updated Adjusted R Square will be 0.2965654924239436
threshold reached...

We can't remove CGPA and further columns

Following features were removed: ['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR ']

We removed 5 features

Training the model to compare the results after our work on multicollinearity

```
In [68]: new_x = df[['CGPA', 'Research']]
new_scaled_x = scaler.fit_transform(new_x)

new_xtrain, new_xtest, new_ytrain, new_ytest = train_test_split(new_x, Y, test
```

```
In [69]: model_training(new_xtrain, new_ytrain, new_xtest, new_ytest, ['CGPA', 'Research'])
model_training(new_xtrain, new_ytrain, new_xtest, new_ytest, ['CGPA', 'Research'])
model_training(new_xtrain, new_ytrain, new_xtest, new_ytest, ['CGPA', 'Research'])
```

-----Linear Regression Model-----

Train MAE: 0.05, Test MAE: 0.05
Train MSE: 0.0, Test MSE: 0.0
Train R-Square: 0.78, Test R-Square: 0.81
Train Adjusted R Square: 0.78, Test Adjusted R Square: 0.81
Train RMSE: 0.06, Test RMSE: 0.07

	Column	coef
0	CGPA	0.185450
1	Research	0.040704

Intercept: -0.8885136249756137

-----Lasso Regression Model-----

Train MAE: 0.05, Test MAE: 0.05
Train MSE: 0.0, Test MSE: 0.0
Train R-Square: 0.78, Test R-Square: 0.81
Train Adjusted R Square: 0.78, Test Adjusted R Square: 0.8
Train RMSE: 0.06, Test RMSE: 0.07

	Column	coef
0	CGPA	0.183957
1	Research	0.037593

Intercept: -0.8739911454415342

-----Ridge Regression Model-----

Train MAE: 0.05, Test MAE: 0.05
Train MSE: 0.0, Test MSE: 0.0
Train R-Square: 0.78, Test R-Square: 0.81
Train Adjusted R Square: 0.78, Test Adjusted R Square: 0.8
Train RMSE: 0.06, Test RMSE: 0.07

	Column	coef
0	CGPA	0.183656
1	Research	0.041346

Intercept: -0.8734748286993076

Observation: After removing collinear features using VIF and using only two features. R2_score and Adjusted_r2 are still the same as before the testing dataset.

Mean of Residuals

```
In [70]: model = LinearRegression()
model.fit(X_train, Y_train)
y_predicted = model.predict(X_test)
```

```
residuals = Y_test - y_predicted
mean_residual = np.mean(residuals)
mean_residual
```

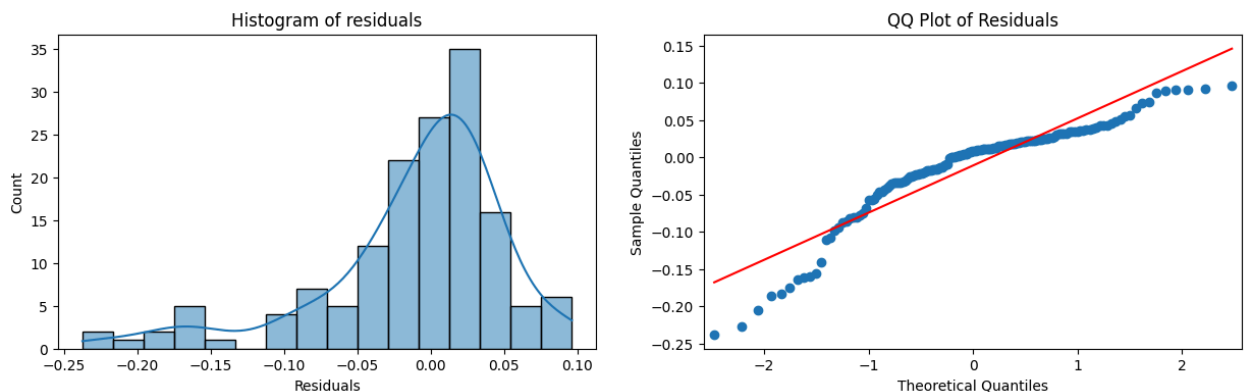
```
Out[70]: np.float64(-0.010793738256654513)
```

Normality of Residuals

```
In [71]: fig, axs = plt.subplots(1, 2, figsize=(15,4))

sns.histplot(residuals, kde=True, ax=axs[0])
axs[0].set_title("Histogram of residuals")
axs[0].set_xlabel("Residuals")

# QQ Plot
sm.qqplot(residuals, line='s', ax=axs[1])
plt.title('QQ Plot of Residuals')
plt.show()
```



Observation: This is evident from the histogram or KDE plot showing a longer tail on the left side (more extreme negative residuals) and the QQ plot deviating from a straight line, which indicates a departure from normality. Left skewness often means the model is overpredicting for some observations (leading to larger negative errors), and this violates the normality assumption of linear regression, potentially affecting inference like confidence intervals or p-values. However, for prediction-focused models like ours, mild non-normality might be less critical if other assumptions (e.g., no multicollinearity, which we've already addressed) hold.

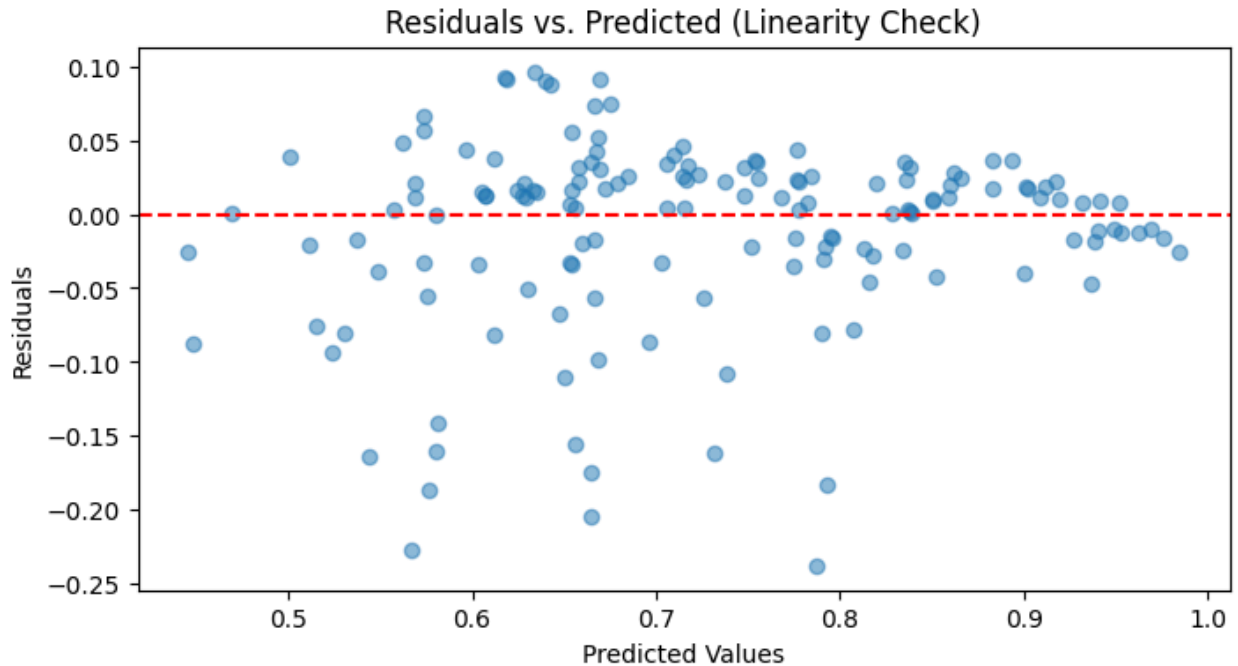
Linearity of variables (no pattern in the residual plot)

```
In [72]: import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4))
plt.scatter(y_predicted, residuals, alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
```

```
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.title('Residuals vs. Predicted (Linearity Check)')
plt.show()

# Qualitative check: Look for random scatter, no curves or funnels
```



Observation: In the Residual Plot there are no systematic patterns that suggest heteroscedasticity in the data. Although there appear to be outliers in the dataset, based on the distance of some residuals from the reference line.

We can use statistical tests to confirm our assumptions.

Goldfeld-Quandt

This test is applicable when we suspect heteroscedasticity is related to a specific independent variable. It splits the data into two or three subgroups based on the values of that variable and compares the variances of the residuals in these subgroups using an F-test

- **Null Hypothesis (H0):** The variances of the error terms are the same across the subgroups.
- **Alternative Hypothesis (Ha):** The variances are different.

Significance level (α): 0.05

```
In [73]: # Performing the Goldfeld-Quandt test to check for Homoscedasticity
name = ['F statistic', 'p-value']
```

```
test = sms.het_goldfeldquandt(Y_train, X_train)
lzip(name, test)
```

```
Out[73]: [('F statistic', np.float64(1.0141968231460807)),
          ('p-value', np.float64(0.46365780964098724))]
```

From the Goldfeld-Quandt test:

- F-Statistic comes out to be 1.01 $\Rightarrow$ minimal difference in variance between groups
- p-value of 0.46 indicates that this difference is not statistically significant at conventional levels of significance. This implies we can accept the null hypothesis of homoscedasticity

Insights:

- Through linear regression modeling on university admission data, we identified key predictors of admission chances after rigorous assumption testing, including checks for heteroscedasticity and multicollinearity. By setting an adjusted R-squared threshold of 0.79, we ensured the model was robust and explanatory without overfitting.
- Multicollinearity analysis led to the removal of five redundant columns (e.g., potentially overlapping scores like GRE and TOEFL, which may correlate highly with CGPA). This streamlined the model to two highly significant, interdependent predictors: CGPA and Research experience.
- These findings reveal that CGPA serves as a foundational indicator of academic readiness, while Research experience adds a layer of differentiation, signaling innovation and depth. Together, they explain a substantial portion of admission variability (adjusted R-squared ~ 0.79), suggesting that students with strong performance in these areas have notably higher chances, even if other factors like test scores are supportive but less pivotal after accounting for correlations.

Actions:

To translate these insights into practical, unbiased implementation, the Jamboori Education Foundation should prioritize a holistic and adaptive approach that balances multiple factors while addressing potential biases. Here are immediate, generalized steps:

- **Enhance Data Integration for Broader Profiles:** Expand the dataset to include diverse indicators of student progression, such as

10th and 12th standard grades, high school performance, and extracurriculars like sports. Launch a data collection initiative (e.g., via user surveys or partnerships with schools) to incorporate these elements, allowing the model to capture long-term academic trajectories and non-traditional strengths. This could be piloted in 3-6 months, with initial testing in varied regions to account for differences—such as how a high CGPA might be commonplace in competitive urban areas but exceptional in underserved rural ones.

- **Apply Feature Engineering to Handle Multicollinearity Inclusively (Important):** Instead of fully removing correlated variables like GRE and TOEFL, develop engineered features that link them thoughtfully. For instance, create composite metrics (e.g., a "problem-solving index" combining GRE/TOEFL scores with CGPA weights, using techniques like principal component analysis to reduce multicollinearity while retaining their real-time assessment value). Implement this in the prediction tool's backend, ensuring outliers—such as students with modest CGPA but strong test scores—are not overlooked. Validate through A/B testing on a subset of users to maintain or exceed the 0.79 adjusted R-squared benchmark.
- **Conduct Regional and Bias Audits:** Roll out the updated tool with built-in audits to evaluate performance across regions, adjusting for contextual factors (e.g., normalizing CGPA significance based on local educational norms). Train foundation staff to use these insights for personalized guidance, emphasizing that no single factor dominates, and monitor for biases against non-research-oriented students or those from diverse backgrounds.

Recommendations (Important to incorporate):

From a broader perspective, these findings highlight the need for an equitable, multifaceted approach to admission predictions—one that empowers students globally while recognizing the limitations of any model. Drawing on educational trends and real-world variability, here are forward-looking strategies to extend your analysis:

- **Develop Improvement Programs with Regional Sensitivity:** While CGPA and research stand out as significant predictors, recommend tailored programs that boost these alongside other skills, customized by region. For example, in areas where high CGPA is rare, offer targeted

tutoring and mentorship; in competitive regions, emphasize differentiation through research or extracurriculars like sports. This could involve collaborations with local schools to create "progression pathways" that track student growth from high school (including 10th/12th scores) onward, providing specialized recommendations like "Build on your strong 12th-grade performance by pursuing research internships to enhance admission odds."

- **Incorporate Holistic and Outlier-Resilient Features:** To avoid ignoring GRE and TOEFL's role in real-time problem-solving, suggest advanced feature engineering, such as interaction terms (e.g., a formula linking GRE scores with research experience: Admission Chance Boost = (GRE Score * 0.4) + (TOEFL Score * 0.3) + (CGPA Adjustment for Correlation)). This creates a "connection link" between variables, preserving their predictive power without multicollinearity pitfalls. Extend this to include high school metrics and sports involvement as proxy indicators of resilience and progression, ensuring the model accommodates outliers—students who excel in tests despite lower CGPA—through ensemble methods like random forests for more nuanced predictions.
- **Promote Inclusive, Adaptive Modeling for Long-Term Impact:** Advocate for ongoing model evolution that integrates emerging data sources, such as global admission trends or alternative assessments (e.g., portfolios for non-traditional learners). Recommend ethical guidelines, like transparency reports on how regional variations affect predictions, to foster trust. For business growth, position the foundation's tool as a comprehensive advisor: offer features like scenario simulations ("If you improve TOEFL by 10 points and add sports leadership, your chances rise by X%"), appealing to a wider audience and potentially increasing user engagement by highlighting diverse paths to success rather than a narrow focus.